

SupportBot — RAG-Powered Customer Service Chatbot

RAG-powered customer service chatbot project documentation generated from the uploaded Markdown file.

Table of Contents

Table of Contents	1
SupportBot — RAG-Powered Customer Service Chatbot	2
Table of Contents	2
Project Overview	2
Phase 1 — Skill Definition	3
Phase 2 — Base Chatbot (CLI + OpenRouter)	3
Phase 3 — Knowledge Base Creation	5
Phase 4 — RAG Implementation	6
Architecture Deep Dive	9
File Reference	11
How to Run	12
Future Improvements	12

SupportBot — RAG-Powered Customer Service Chatbot

A terminal-based customer service chatbot that uses Retrieval-Augmented Generation (RAG) to answer company policy questions by searching a curated knowledge base before generating responses.

Table of Contents

- 1. [Project Overview](#project-overview)
- 2. [Phase 1 — Skill Definition](#phase-1--skill-definition)
- 3. [Phase 2 — Base Chatbot](#phase-2--base-chatbot-cli--openrouter)
- 4. [Phase 3 — Knowledge Base Creation](#phase-3--knowledge-base-creation)
- 5. [Phase 4 — RAG Implementation](#phase-4--rag-implementation)
- 6. [Architecture Deep Dive](#architecture-deep-dive)
- 7. [File Reference](#file-reference)
- 8. [How to Run](#how-to-run)
- 9. [Future Improvements](#future-improvements)

Project Overview

What this project does

SupportBot is a question-answering chatbot for customer service. A user types a question in the terminal (e.g. "What is the return policy?" or "Can I change my delivery address?"), and the bot responds with accurate information drawn from the company's internal knowledge base.

Why RAG?

A standard chatbot relies only on its system prompt. If the prompt is too large, the model loses focus; if too small, the model doesn't have enough information. RAG solves this by:

- 1. Storing the knowledge base as vector embeddings (semantic search index)
- 2. On each question, finding the most relevant policy documents
- 3. Injecting only those documents into the prompt as context

This keeps the prompt small, relevant, and grounded in actual policy — reducing hallucinations.

Tech Stack

| Component | Technology |

|---|---|

| Language | Python 3.12 |

LLM API	OpenRouter (model-agnostic — any model works)
Embeddings	OpenRouter / text-embedding-3-small
Vector Database	ChromaDB (persistent, local, no server)
HTTP Client	httpx
Config	python-dotenv / .env
Virtual Env	uv / venv

Phase 1 — Skill Definition

What happened

A `skill.md` file was created to define the blueprint for a customer service chatbot. This is an opencode skill — it tells the AI assistant how to build, configure, and improve customer service chatbots.

Why this matters

The skill acts as the **specification document** for the entire project. Instead of guessing features, every decision is traced back to the skill:

- What knowledge areas should the bot cover? → Orders, accounts, products, returns, shipping, billing, policies
- How should it behave? → Polite, empathetic, solution-first, transparent
- When should it escalate? → 8 defined triggers (legal threats, 3+ failed resolutions, etc.)
- How should it refuse? → Hard safety rules including a strict refund policy (no processing, immediate escalation)

Key design decisions from the skill

1. **Refund policy:** The bot is explicitly forbidden from processing, discussing, or promising refunds. It escalates immediately with a single line. This was added after testing revealed the bot would walk customers through refund steps rather than declining.
2. **Scope boundaries:** The bot must refuse anything outside customer service (medical, legal, financial advice) politely.
3. **Escalation threshold:** After 3 repeated questions on the same topic, the bot hands off to a human.

Files produced

- `skill.md` — human-readable skill definition
 - `~\.opencode\skills\customer-service\SKILL.md` — registered opencode skill
-

Phase 2 — Base Chatbot (CLI + OpenRouter)

What happened

A working terminal chatbot was built before adding any RAG. This established the core loop:

1. Read user input
2. Build messages array (system prompt + history + new question)
3. Send to OpenRouter chat completions API
4. Print response
5. Append to history

Code structure: main.py

```
main.py
#### Imports & config
■ #### os, httpx, dotenv
■ #### Load .env (API_KEY, MODEL)
■ #### Define API_URL
■ #### BASE_SYSTEM_PROMPT (string)
■ #### Identity & role
■ #### Core knowledge areas
■ #### Tone & language rules
■ #### Response format
■ #### Hard safety rules
■ #### Refund strict rule
■ #### Escalation triggers
■ #### Escalation handoff
■ #### build_system_prompt(context)
■ #### Prepends RAG context before the base prompt
■ #### ask_bot(user_message, history, rag_engine)
■ #### Retrieve context (if RAG available)
■ #### Build messages array
■ #### POST to OpenRouter
■ #### Return response text
■ #### main()
■ #### Initialise RAG engine
■ #### REPL loop (input → ask_bot → print → history.append)
■ #### Commands: quit, reset
```

The prompt engineering challenge

The system prompt went through several iterations:

Attempt 1 (basic): "Be polite, help with orders." → Bot gave vague answers.

Attempt 2 (structured): Added 6 sections (identity, knowledge, tone, format, safety, escalation). → Better but still handled refunds.

Attempt 3 (refund block): Added explicit "do NOT process refunds" rule with exact escalation phrase. → Bot still asked for order numbers before refusing.

Attempt 4 (final): Added "do not ask for order numbers, reasons, or any details" — immediate escalation. → Works correctly.

Why httpx (not requests)

- Native async support (future-proof for web server)
- Built-in timeout handling
- HTTP status code exceptions via `raise_for_status()`
- Same API as `requests` for simple calls

Conversation history

- Stored as a list of {"role": "user"/"assistant", "content": "..."} dicts
 - Capped at 40 messages (20 exchanges) to avoid token limits
 - Cleared on reset command
-

Phase 3 — Knowledge Base Creation

What happened

Raw CSV data (~580 records) was cleaned and restructured into a proper knowledge base of 18 markdown files.

Source data analysis

The CSV contained 5 sections:

| Section | Content | Quality |

|---|---|---|

| A — Live Chat Transcripts | 8 real customer-agent conversations | Useful for extracting policies, but conversational format was noisy |

| B — Email Support Tickets | 7 tickets (refunds, shipping, logins, promo codes, complaints) | Policy-rich but mixed with personal customer data |

| C — Internal FAQ Notes | Agent handoff notes — shorthand policies | **Most valuable section** — direct policy statements |

| D — Call Centre Brief Notes | Agent shorthand summaries | Contained edge cases (empty boxes, wrong items, damaged packaging) |

| E — Duplicates & Contradictions | Auto-flagged inconsistencies | Used as review checklist |

Cleaning methodology

Step 1 — Extract policies, discard conversations

The chat transcripts and emails contained personal customer information (names, emails, order numbers) and conversational filler. Only the policy outcome was kept. For example:

```
Raw: "Customer: I want to return a dress I bought"
"Agent: Our return window is 30 days so you're fine"
"Customer: it says 'return not eligible'"
"Agent: Let me override it for you manually"

Clean: "Return window: 30 days from delivery"
"If return is not eligible in system, agent can manually override"
```

Step 2 — Normalise formats

Order numbers appeared in two formats: ORD-9912 and ORD88210. All were normalised to ORD-XXXXX format in the KB (the chatbot handles both input formats).

[illegible]

Code structure: rag.py

```
rag.py
#### Constants
# #### EMBED_MODEL = "openai/text-embedding-3-small"
# #### EMBED_URL = "https://openrouter.ai/api/v1/embeddings"
# #### CHUNK_SIZE = 600 characters
# #### CHUNK_OVERLAP = 100 characters
#### load_kb_files()
# #### Recursively finds all *.md in kb/
# #### For each file: read, split into chunks
# #### Returns list of {"text", "source", "topic", "file"}
#### split_into_chunks(text, source)
# #### Splits by line boundaries (not mid-word)
# #### Each chunk = ~600 chars
# #### Overlap = ~100 chars (last few lines of previous chunk)
# #### Prevents context loss at chunk boundaries
#### get_embedding(text)
# #### POST to OpenRouter embeddings API
# #### Returns list of 1536 floats
# #### Raises error if API key is placeholder
#### class RAGEngine
#### __init__()
# #### Creates/opens ChromaDB at chroma_db/
# #### Gets or creates collection "customer_service_kb"
```

```

■ ■■■ Calls _ensure_indexed()
■■■ _ensure_indexed()
■ ■■■ If collection is empty: load all chunks, embed each, store
■ ■■■ If collection has data: skip (cached)
■ ■■■ Progress printed to terminal during first run
■■■ retrieve(query, top_k=3)
■■■ Embeds the query
■■■ Queries ChromaDB with cosine similarity
■■■ Returns top_k documents as formatted string
■■■ Returns "" on error (graceful fallback)

```

Chunking strategy

Documents are split into 600-character chunks with 100-character overlap. Why:

- **600 chars** — roughly 2-3 paragraphs, enough for a complete policy point
- **100 overlap** — ensures a sentence spanning two chunks isn't lost
- **Line-boundary aware** — splits at newlines, not mid-word or mid-sentence

kb/ (18 files) → load_kb_files() → 21 chunks

Embedding model selection

text-embedding-3-small was chosen because:

- 1536 dimensions (good balance of precision vs cost)
- Available via OpenRouter (no extra API key needed)
- Cheaper than text-embedding-3-large (256 dimensions saved, ~80% lower cost)
- Sufficient for document-level retrieval (not needing fine-grained search)

Why ChromaDB

Feature	ChromaDB	FAISS	Pinecone
Server required	No	No	Yes
Persistent storage	Yes (SQLite-based)	Manual	Yes
Metadata filtering	Built-in	Manual	Built-in
Setup time	1 pip install	1 pip install	Account + API key
Free tier	Unlimited local	Unlimited	Limited

ChromaDB was chosen because it's the simplest to set up (pure Python, no server) while providing persistence and metadata filtering out of the box.

Modified main.py additions

```

from rag import RAGEngine # New import

BASE_SYSTEM_PROMPT = ""..." # Renamed from SYSTEM_PROMPT

def build_system_prompt(context: str) -> str:
    # Prepends RAG context if available
    if context:
        return f"{BASE_SYSTEM_PROMPT}\n\n=== RELEVANT COMPANY POLICIES ===\n{context}\n\n=== END

```

```
POLICIES ===""
return BASE_SYSTEM_PROMPT

def ask_bot(user_message: str, history: list, rag: RAGEngine | None = None) -> str:
    context = rag.retrieve(user_message) if rag else "" # New: retrieve
    system_prompt = build_system_prompt(context) # New: inject
    ...

def main():
    rag = RAGEngine() # New: init on startup
    ...
    response = ask_bot(user_input, history, rag) # New: pass rag
```

Graceful degradation

If the API key is invalid or has no credits, RAG still works:

- `RAGEngine()` might fail during indexing → `rag` is set to `None`
- `rag.retrieve("...")` returns `""` on error
- `build_system_prompt("")` returns the base prompt unchanged
- The bot still functions as a non-RAG chatbot

Architecture Deep Dive

Data Flow Diagram



```

■ ■ ■
■ ▼ ■
■ Top 3 chunks: ■
■ [return-policy] "30 days from delivery..." ■
■ [initiate-return] "Go to My Orders..." ■
■ [refund-timeline] "2-3 business days..." ■
■ ■ ■
■ ▼ ■
■ build_system_prompt(context) ■
■ ■ ■
■ ▼ ■
■ POST /api/v1/chat/completions ■
■ model: openai/gpt-4o ■
■ messages: [system prompt + context, history, question] ■
■ ■ ■
■ ▼ ■
■ SupportBot: "Our return policy allows returns ■
■ within 30 days of delivery..." ■

```

Token Usage Breakdown

Per query (estimated):

Component	Tokens
System	1
User	1
Product	1
Service	1
Feature	1
Benefit	1
Objection	1
Testimonial	1
Call to Action	1
Header	1
Footer	1
Image	1
Video	1
Form	1
Link	1
Text	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1
Quote	1
Section	1
Paragraph	1
List	1
Table	1
Figure	1
Code	1

|-|-|-|-|-|

| Base system prompt | ~400 |

| RAG context (3 chunks) | ~300-500 |

| Conversation history (avg 3 exchanges) | ~200 |

| User question | ~20 |

| **Total sent** | ~920-1120 |

| Model response (max tokens=500) | ~500 |

| Total per query | ~1420-1620 |

With `max_tokens=500`, each query stays well under free-tier limits.

Why This Works for Customer Service

Customer service queries fall into predictable categories. The 18 KB files cover the vast majority of real questions:

| Question | Retrieved Files |

$$\begin{array}{c} | \quad \quad | \quad \quad | \\ \text{---} \text{---} \text{---} \text{---} \text{---} \end{array}$$

```
| "Where is my order?" | orders/status-check.md, shipping/delivery-times.md |
```

| "I want a refund" | Escalated (strict rule), but would retrieve returns/refund-timeline.md |

| "Can I change my address?" | orders/modification.md |

| "Do you have a physical store?" | [faq/general.md](#) |

| "My promo code doesn't work" | policies/promo-codes.md |

| "How do I reset my password?" | account/password-reset.md |

File Reference

main.py — Entry point

| Function | Purpose |

---|---

| build_system_prompt(context) | Prepends RAG context to the base system prompt |

| ask_bot(message, history, rag) | Retrieves context, calls OpenRouter, returns response |

| main() | Initialises RAG, runs the REPL loop |

rag.py — RAG Engine

| Function / Class | Purpose |

---|---

| load_kb_files() | Reads kb/ directory, returns chunked documents |

| split_into_chunks(text, source) | Splits markdown into 600-char overlapping chunks |

| get_embedding(text) | Calls OpenRouter embeddings API |

| RAGEngine | Manages ChromaDB, auto-indexes on first run, retrieves on query |

kb/ — Knowledge Base (18 files)

| File | Key policies |

---|---

| returns/return-policy.md | 30-day window, non-returnable items, wrong/damaged item policy |

| returns/initiate-return.md | Online portal vs agent-assisted, store credit |

| returns/refund-timeline.md | 2-3 days warehouse, 5-10 days bank, checking refund status |

| orders/status-check.md | Tracking link, email troubleshooting, order statuses |

| orders/cancellation.md | Before/after shipping rules, urgent flag |

| orders/modification.md | Address change, adding items, gift wrapping |

| shipping/delivery-areas.md | COD locations (Accra, Kumasi, Takoradi), international duties |

| shipping/delivery-times.md | Same-day cutoff 2PM, no specific date promises |

| account/password-reset.md | 30-min link expiry, manual resend, locked accounts |

| account/update-details.md | Phone/email/address updates |

| account/loyalty.md | 1pt/GHS1, minimum 50pts redeem, no referral programme |

| products/availability.md | Waitlist, authenticity, size guide |

| products/specifications.md | Dimensions, colours, screen calibration |

policies/warranty.md	Damaged/wrong item photo within 7 days, replacement or refund
policies/promo-codes.md	One per order, expiry check, FRESH15/SAVE20
policies/payment.md	Card, MoMo, Vodafone, COD, BNPL, MoMo failure fix
faq/general.md	Online only, bulk orders → b2b@, invoices, gift wrap, social media
escalation/triggers.md	Legal threats, 3+ failed resolutions, media threats

chroma_db/ — Vector database (auto-generated)

Contains ChromaDB's persistent SQLite storage. On first run, 21 vectors are stored here. Subsequent runs load instantly.

How to Run

```
# Ensure .env has your OpenRouter key
# OPENROUTER_API_KEY=sk-or-v1-...
# OPENROUTER_MODEL=openai/gpt-4o

# Activate virtual environment
.venv\Scripts\python.exe main.py

# Commands:
# Type your question and press Enter
# Type 'quit' or 'exit' to stop
# Type 'reset' to clear conversation history
```

First run — will take longer (~30 seconds) as it indexes all 21 chunks. Each chunk calls the OpenRouter embeddings API.

Subsequent runs — instant. ChromaDB loads from local cache.

Future Improvements

| Feature | Reason |

|---|---|

| Web interface (FastAPI + HTML) | Move beyond terminal |

| Multiple embedding models | Compare accuracy vs cost |

| Hybrid search (keyword + vector) | Catch exact matches better than semantic search alone |

| Feedback loop (thumbs up/down) | Track which answers were helpful |

| Live KB reload (watch files) | Update KB without restarting |

| Async streaming | Show tokens as they arrive (faster UX) |

| Document update API | Add/remove KB entries at runtime |